

GPU Info Script Explanation

Overview

This script uses Python and ctypes to talk directly to Windows (DXGI) and retrieve GPU information such as name, total VRAM, and available VRAM.

ctypes

ctypes allows Python to call low-level Windows APIs written in C. This is how we access DirectX (DXGI).

Structures

The script defines structures that match Windows data formats. These act as containers that Windows fills with GPU information.

COM / DXGI

DXGI is part of DirectX and is used to enumerate GPUs. COM interfaces are used to interact with these system components.

VTable Access

The script manually calls COM methods using a vtable. This is a low-level technique where functions are accessed by index.

Main Logic

The script loops through all GPU adapters, retrieves descriptions and memory info, filters valid GPUs, and stores results in a dictionary.

Full Source Code

```
import ctypes

DXGI_MEMORY_SEGMENT_GROUP_LOCAL = 0

VENDOR_AMD      = 0x1002
VENDOR_NVIDIA   = 0x10DE
VENDOR_INTEL     = 0x8086

class DXGI_QUERY_VIDEO_MEMORY_INFO(ctypes.Structure):
    _fields_ = [
        ("Budget", ctypes.c_uint64),
        ("CurrentUsage", ctypes.c_uint64),
        ("AvailableForReservation", ctypes.c_uint64),
        ("CurrentReservation", ctypes.c_uint64),
    ]

class DXGI_ADAPTER_DESC(ctypes.Structure):
    _fields_ = [
        ("Description", ctypes.c_wchar * 128),
        ("VendorId", ctypes.c_uint),
        ("DeviceId", ctypes.c_uint),
        ("SubSysId", ctypes.c_uint),
        ("Revision", ctypes.c_uint),
        ("DedicatedVideoMemory", ctypes.c_size_t),
        ("DedicatedSystemMemory", ctypes.c_size_t),
        ("SharedSystemMemory", ctypes.c_size_t),
        ("AdapterLuid", ctypes.c_byte * 8),
    ]

IID_IDXGIFactory1 = (ctypes.c_byte * 16)(
    0x78, 0xae, 0x0a, 0x77,
    0x6f, 0xf2,
    0xba, 0x4d,
    0xa8, 0x29,
    0x25, 0x3c, 0x83, 0xd1, 0xb3, 0x87
)

IID_IDXGIAdapter3 = (ctypes.c_byte * 16)(
    0xA4, 0x67, 0x59, 0x64,
    0x92, 0x13,
    0x10, 0x43,
    0xA7, 0x98,
    0x80, 0x53, 0xCE, 0x3E, 0x93, 0xFD
)

def _vtable_fn(obj, slot, restype, *argtypes):
    vtable = ctypes.cast(obj, ctypes.POINTER(ctypes.c_void_p))
    fn_ptr = ctypes.cast(vtable[0], ctypes.POINTER(ctypes.c_void_p))[slot]
    return ctypes.cast(fn_ptr, ctypes.CFUNCTYPE(restype, ctypes.c_void_p, *argtypes))

def _release(obj):
    _vtable_fn(obj, 2, ctypes.c_ulong)(obj)

def _query_interface(obj, iid, out):
    fn = _vtable_fn(obj, 0, ctypes.c_long,
        ctypes.POINTER(ctypes.c_byte * 16),
        ctypes.POINTER(ctypes.c_void_p))
    return fn(obj, iid, out)

def get_dxgi_gpu_info() -> dict:
```

```

ctypes.windll.ole32.CoInitialize(None)

factory = ctypes.c_void_p()
if ctypes.windll.dxgi.CreateDXGIFactory1(ctypes.byref(IID_IDXGIFactory1), ctypes.byref(factory)) != 0:
    ctypes.windll.ole32.CoUninitialize()
    return {}

enum_adapters = _vtable_fn(factory, 12, ctypes.c_long,
                           ctypes.c_uint,
                           ctypes.POINTER(ctypes.c_void_p))

gpus = {}
index = 0

while True:
    adapter1 = ctypes.c_void_p()
    if ctypes.c_uint32(enum_adapters(factory, index, ctypes.byref(adapter1))).value != 0:
        break

    adapter3 = ctypes.c_void_p()
    qi_hr = _query_interface(adapter1, ctypes.byref(IID_IDXGIAdapter3), ctypes.byref(adapter3))
    _release(adapter1)

    if ctypes.c_uint32(qi_hr).value != 0:
        index += 1
        continue

    desc = DXGI_ADAPTER_DESC()
    mem_info = DXGI_QUERY_VIDEO_MEMORY_INFO()

    _vtable_fn(adapter3, 8, ctypes.c_long,
               ctypes.POINTER(DXGI_ADAPTER_DESC))(adapter3, ctypes.byref(desc))

    _vtable_fn(adapter3, 14, ctypes.c_long,
               ctypes.c_uint,
               ctypes.c_uint,
               ctypes.POINTER(DXGI_QUERY_VIDEO_MEMORY_INFO))(
        adapter3, 0, DXGI_MEMORY_SEGMENT_GROUP_LOCAL, ctypes.byref(mem_info))

    total = desc.DedicatedVideoMemory // (1024 ** 3)
    free = (mem_info.Budget - mem_info.CurrentUsage) // (1024 ** 3)

    if total > 0 and desc.VendorId in (VENDOR_AMD, VENDOR_NVIDIA, VENDOR_INTEL) and free <= total:
        gpus[index] = {
            "name": desc.Description,
            "totalMem": total,
            "freeMem": free,
        }

    _release(adapter3)
    index += 1

_release(factory)
ctypes.windll.ole32.CoUninitialize()
return gpus

print(get_dxgi_gpu_info())

```